

command in a terminal:

```
go version
```

The result will display the Go version number.

Hello World: A sample Go program

A Go program basically consists of the following components:

- Package declaration
- Package import
- Functions and variables
- Statements and expressions
- Comments

In our sample program, we will try to use all the components. Since we have already created our workspace `$HOME/dev`, where we are going to create all our programs, let's add the following code to any text editor:

```
package HelloWorld //
package declaration (every GoProgram must start with it)
import "fmt"
//importing standard package for formatting input and output
// first sample program - comment always starts with two / and
it is ignored by Go Compiler
func main() {
    fmt.Println("Hello World!") //calling
the method of the package fmt which prints line
}
```

Let's save it as `HelloWorld.go` in the workspace directory. Enter the following command in a terminal:

```
cd $GOPATH //change the
current directory to our workspace directory
go run HelloWorld.go //run the code
stored in the file "HelloWorld.go"
```

The result will be displayed in the terminal as 'Hello world!'. The `Go run` command reads the files (separated by spaces), compiled them, converted them into an executable saved in a temporary directory and then ran the program.

Data types, variables and constants

Data types are used to describe what type of data will be stored and how much space will be acquired to store that data. Go comes with several built-in data types.

Variables are used to give a name to the storage area. The name of a variable can contain letters, digits, and the underscore character. But it must start with either a letter or an underscore. Go is a case-sensitive language; therefore, upper case and lower case letters are distinct.

A variable definition is used to tell the compiler where and how much space to create for the variable. This definition

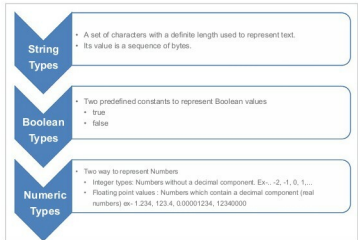


Figure 1: Data types in Go

is composed of a data type and contains a list of one or more variables of that type.

The syntax is `var var_list optional_data_type;`

Here, `optional_data_type` can be of any Go data type, i.e., byte, int, float32, complex64, Boolean or any user-defined object. And `var_list` may consist of one or more variables separated by commas.

Some valid declarations are shown here:

```
var i, j, k int;
var a, c byte;
var b, sal float32;
d = 42;
```

Variables have two types of declarations.

- 1) **Static type:** The compiler knows about the type and name of the variable at the compilation time only.
- 2) **Dynamic type:** The compiler needs to interpret the type of variable based on the value passed to it.

Constants are basically variables whose values cannot be changed during the lifetime of the program. We can create constants in the same way we declare variables, but instead of `var`, we should use the `const` keyword.

```
const x string = "Hello World"
```

Flow control

The decision-making structure: When program execution depends on one or more conditions, then we should use decision-making structures. These enable us to evaluate the condition at the point of execution, and only execute a statement or statements if the condition is true or false.

The basic structure is shown in Figure 2.

Here is a sample program to print odd and even numbers between 1 and 50.

```
func main() {
    for i := 1; i <= 50; i++ {
        if i % 2 == 0 {
```